

ALERT 4.0 GOVERNMENT - Architecture & Workflow Documentation

This document outlines the detailed system architecture, database implementations, and complete workflows for the ALERT 4.0 GOVERNMENT platform.

1. Microservices Architecture

The backend is structured as a distributed microservices system managed by Netflix Eureka.

Microservice	Internal URL (Docker)	Public API Gateway URL	Role & Responsibilities
API Gateway	<code>http://api-gateway:8081</code>	<code>http://localhost:8081</code>	Centralized routing, CORS management, and reverse proxy for all external frontend requests.
Auth Service	<code>http://auth-service:8082</code>	<code>/api/v1/auth/**</code>	Handles user authentication, token generation, and role-based access control (RBAC).
User Service	<code>http://user-service:8083</code>	<code>/api/v1/users/**</code>	Manages user profiles, role hierarchy (Village -> State), and state geographical reference data.
Incident Service	<code>http://incident-service:8083</code>	<code>/api/v1/incidents/**</code>	The core engine. Tracks disaster incidents, stores incident media, and powers the escalation timeline.
Alert Service	<code>http://alert-service:8084</code>	<code>/api/v1/alerts/**</code>	Manages public broadcasts and internal alert propagation.
Approval Service	<code>http://approval-service:8085</code>	<code>/api/v1/approvals/**</code>	Queues and manages Collector broadcasts pending State Admin authorization.
Analytics Service	<code>http://analytics-service:8088</code>	<code>/api/v1/analytics/**</code>	Aggregates incident resolution times, response trends, and regional performance stats.
AI Service (Java)	<code>http://ai-service:8089</code>	<code>/api/v1/ai/**</code>	Secure proxy and data formatter for the internal Python AI pipeline.

AI Service (Py)	<code>http://alertgov-ai:8000</code>	<i>Internal Only</i>	Llama 3 powered intelligence. Predicts disaster severity and parses weather data for advisories.
------------------------	--------------------------------------	----------------------	--

2. Database Infrastructure

The system employs a polyglot persistence architecture, choosing the optimal database type for specific data workloads.

Oracle Database (Relational)

- **Host:** `oracle-db:1521:XE`
- **Purpose:** ACID compliant storage for highly structured, relational data.
- **Used By:** `auth-service, user-service, incident-service, analytics-service, ai-service, approval-service`.
- **Key Tables:**
 - `USERS, ROLES` - Strict hierarchy management.
 - `INCIDENTS` - Relational tracking of incident state, severity, and assigned district.
 - `APPROVALS` - Transactional state management ensuring broadcasts aren't duplicated.

MongoDB Atlas (NoSQL Document Store)

- **Host:** `cluster0.7efvpxm.mongodb.net`
- **Purpose:** High-speed, schema-less storage for unstructured media and binary assets.
- **Used By:** `incident-service`.
- **Key Collections:**
 - `alertgov_media` - Stores base64-encoded incident images and video attachments captured by the Village EOC operators.

3. Detailed Workflows

A. Real-Time Incident Escalation Workflow

1. **Reporting (Village EOC):** A local operator creates an incident. Text data is saved to Oracle DB; attached images are saved to MongoDB Atlas.
2. **AI Verification (Taluk Office):** The Taluk officer reviews the incident. The frontend calls `/api/v1/ai/predict/{id}`. Llama 3 analyzes the description, categorizes the severity, and flags if escalation is required.
3. **Escalation (District -> Collector):** The incident is sequentially pushed up the chain (`PUT /api/v1/incidents/{id}/status`). The React UI tracks completion via the `EscalationTracker` component, rendering precise timestamps for each stage.

B. Autonomous Weather Advisory Workflow

1. **Trigger:** State Admin accesses the AI Prediction Dashboard.
2. **Fetch:** The Python AI Service (`/ai/weather-advisory`) pulls live API weather data for Tamil Nadu.
3. **Analyze:** Llama 3 analyzes the live precipitation and wind-speed data to formulate a custom advisory.
4. **Fallback:** If the Llama 3 model takes longer than 3 seconds to spin up, the Python service intercepts the timeout and instantly returns a simulated fallback response to prevent UI hangs.
5. **Dispatch:** The State Admin reviews the AI draft and dispatches it immediately to the Collector queue.

C. Broadcast Approval Workflow

1. **Drafting (Collector):** The Collector creates a localized broadcast (`POST /api/v1/alerts/broadcast`).
2. **Queuing (Approval Service):** The broadcast is placed into a "Pending" state in the Oracle DB, visible on the State Admin's dashboard (`GET /api/v1/approvals/broadcasts/pending`).
3. **Authorization (State Admin):** The State Admin clicks "Approve".
4. **Publication:** The status changes to "Live", and it is pushed out via the API Gateway to all public-facing disaster portals.